

Reasoning traces/Chain of Thought (CoT)

Consider a task

- described by prompt $\backslash \mathbf{x}$ (a sequence)
- with response $\backslash \mathbf{y}$ (a sequence)

The most direct solution of the task can be described by the following sequence describing the LLM's computation

$\backslash \mathbf{x}, \backslash \mathbf{y}$

If the task is sufficiently complicated

- e.g., is best solved by multi-step reasoning

it has been shown that the chances of generating a better $\hat{y}$ (sequence) answer are improved by "Chain of Thought" reasoning.

CoT summary

Given prompt

$\backslash \mathbf{x}_{(1:\bar{T})}$

rather than *immediately* producing response

$\backslash \mathbf{y}_{(1:\bar{T})}$

giving computation trace

$\backslash \mathbf{x}_{(1:\bar{T})}, \backslash \mathbf{y}_{(1:T)}$

an LLM is trained to think "Step by Step"

- creating a sequence of steps
- the Chain of Thought ("reasoning" trace) $\backslash \mathbf{rat}$
- enumerating sequential steps of a process that produces the response

$$\backslash \mathbf{rat} = [\backslash \mathbf{rat}_{(1)}, \dots, \backslash \mathbf{rat}_{(\text{num_thoughts})}]$$

- where each $\backslash \mathbf{rat}_{\backslash \text{tp}}$ is a thought represented as multi-token sequence

resulting in trace

$\backslash \mathbf{x}, \backslash \mathbf{rat}, \backslash \mathbf{y}$

Thus, the model computes response $\backslash y$ as
 $\backslash pr \backslash y \mid \backslash x, \backslash rat$
rather than directly as

$$\backslash pr \backslash y \mid \backslash x$$

The reasoning trace $\backslash rat$

- *conditions* $\backslash y$
- on the reasoning steps
- improving Performance

More formally, the LLM's "thought process" can be represented as a concatenation of

- prompt $\backslash \mathbf{x}$
- the *reasoning trace* $\backslash \mathbf{rat}$
 - a sequence of *thoughts* ordered linearly: the Chain of Thought

$$\backslash \mathbf{rat} = [\backslash \mathbf{rat}_{(1)}, \dots, \backslash \mathbf{rat}_{(\text{num_thoughts})}]$$

- response $\backslash \mathbf{y}$

Note that each thought $\backslash \mathbf{rat}_{\text{tp}}$ is a sequence of tokens.

A model that reasons: "Think before speaking"

Rather than immediately outputting the "final" response

- we give the model the ability to "think"
 - create reasoning traces
- These thoughts can produce
 - multiple "preliminary" responses
- before returning the single "final" response

The thinking process can process can involve different strategies

- sequential
- parallel
- search/tree

Note

Syntactically:

The thoughts of the sequential strategy are easiest to display

- just a sequence
- parallel or tree strategies involve structure which complicates the display

By convention, thoughts are bracketed by the "think" delimiter

`\bthink`

Thinking step-by-step, the first step is . . .

`\ethink`

This gives the option

- to suppress the reasoning trace when the user only wants to see the response

Here is an example.

The prompt is

Prove to me that the residuals of Linear Regression are not correlated with the independent variables

Note

- the reasoning trace is truncated (ran out of credits on HF before I could write the entire file)
- output: highlight the first words of each paragraph
 - indicates
 - thinking step by step
 - revision, etc

```
In [3]: # Usage
prompt = "Prove to me that the residuals of Linear Regression are not correlated
with the independent variables"
print(prompt)
print("\n")
r1_out_file = 'r1_prove_residuals_orthogonal.txt'

formatted_content = format_file_content(r1_out_file)

# formatted_content = format_file_content('output.txt')
print(formatted_content)
```

Prove to me that the residuals of Linear Regression are not correlated with the independent variables

<think>

****Okay****, so I need to prove that the residuals from a linear regression are not correlated with the independent variables. Hmm, let me start by recalling what residuals are. They're the differences between the actual observed values of the dependent variable and the values predicted by the model. Right, so residuals = $y - \hat{y}$.

****Now****, in linear regression, we have the model $y = X\beta + \varepsilon$, where X is the matrix of independent variables, β is the coefficient vector, and ε is the error term. The residuals are estimates of these errors, right? But wait, the errors are assumed to be uncorrelated with the independent variables in the classical linear regression model. Is that part of the Gauss-Markov assumptions? Yeah, I think one of the assumptions is that the error term has zero conditional mean, meaning $E(\varepsilon|X) = 0$. That implies that the errors are uncorrelated with the independent variables. But how does that translate to the residuals?

****Wait****, the residuals are not the same as the error terms. The residuals are calculated after estimating β , so they're based on the estimated coefficients. So even if the true errors are uncorrelated with X , we need to show that the residuals from the estimated model are also uncorrelated with X .

****Let me think about the normal equations****. In ordinary least squares (OLS), the estimates of β are obtained by solving $X'(y - X\beta) = 0$. That's the first-order condition for minimizing the sum of squared residuals. So, $X'\hat{y} = X'y$, which implies that $X'(y - \hat{y}) = 0$. But $y - \hat{y}$ is the vector of residuals, let's call that e . So $X'e = 0$.

****Ah****, so if $X'e = 0$, that means that the dot product of each column of X with the residuals e is zero. In other words, each independent variable (each column in X) is orthogonal to the residuals. If they're orthogonal, then their co

variance should be zero, right? Because covariance is a measure of linear association, and if they're orthogonal, there's no linear relationship.

****But wait****, covariance is about being linearly related, and orthogonality in the geometric sense. If the inner product is zero, then the covariance is zero only if the variables have zero mean? Wait, no. Let me recall that covariance between two variables is $E[(X -$

Training an LLM to reason

In order to produce an LLM that "reasons" given prompt $\backslash x$

- we train it to produce responses

$\backslash rat, \backslash y$

- rather than the direct responses

$\backslash y$

by training it on examples of the form

$\backslash x, \backslash rat, \backslash y$

- rather than

$\backslash x, \backslash y$

The cost of producing examples with reasoning traces

$\backslash \mathbf{x}, \backslash \mathbf{rat}, \backslash \mathbf{y}$

can be substantial.

Often, it involves asking a human to adapt a training set with examples

$\backslash \mathbf{x}, \backslash \mathbf{y}$

by creating the reasoning traces $\backslash \mathbf{rat}$

Reducing the cost of creating examples with reasoning traces

Removing the human from the training data production process is highly desirable (cost reduction).

Boot-strapping a training dataset from an existing model is one solution.

One method of boot-strapping:

Ask a strong, non-reasoning LLM $\backslash \text{model}^{\text{non-reasoner}}$ to solve task

- given $\backslash \text{x}^{\text{ip}}$
- "think step by step" (create $\backslash \text{rat}^{\text{ip}}$)
- produce response $\backslash \text{y}^{\text{ip}}$

Note that Chain of Thought ("think step by step") reasoning

- seems to emerge from the natural training examples used to train ~~model~~^{non-reasoner}
- does *not* require explicit training examples with reasoning traces
- hence, the boot-strapping process is well grounded
 - non-circular: the first model is trained without reasoning traces

Alternatively:

Ask an existing reasoning model $\backslash \text{model}^{\text{reasoner}}$ to solve task

- given $\backslash \text{x}^{\text{ip}}$
- produce response $\backslash \text{rat}^{\text{ip}}, \backslash \text{y}^{\text{ip}}$

This effectively bootstraps from existing reasoner $\backslash \text{model}^{\text{reasoner}}$

- base case: *someone* needs to (manually) create the dataset to train the first reasoner

Both these approaches creates a machine-generated example

$\langle \backslash \textcolor{red}{x}^{\textcolor{red}{ip}}, \backslash \textcolor{red}{rat}^{\textcolor{red}{ip}}, \backslash \textcolor{red}{y}^{\textcolor{red}{ip}} \rangle$

that can be used to train a new, reasoning LLM

Distillation

The latter approach describes a use of the process called *Distillation*

We start by

- training model $\backslash \text{model}^{\text{teacher}}$ to solve a task
- given training dataset $\langle \backslash \mathbf{X}, \backslash \mathbf{y} \rangle$
 - examples of how the task maps input $\backslash \mathbf{x}^{\text{ip}}$ to response $\backslash \mathbf{y}^{\text{ip}}$ for $1 \leq i \leq m$

We can then train a model $\backslash \text{model}^{\text{student}}$

- to *mimic* $\backslash \text{model}^{\text{teacher}}$
- using $\backslash \text{model}^{\text{teacher}}$
 - on new inputs $\backslash \dot{X}^{\text{ip}}$
 - to produce outputs $\backslash \dot{y}^{\text{ip}}$
 - creating dataset $\langle \backslash \dot{X}, \backslash \dot{y} \rangle$
- and training $\backslash \text{model}^{\text{student}}$ on $\langle \backslash \dot{X}, \backslash \dot{y} \rangle$

Note that

- $\backslash \text{model}^{\text{student}}$ does not directly learn the task
- it only learns to mimic $\backslash \text{model}^{\text{teacher}}$
- inheriting all the flaws and limitations of the teacher

$\backslash \text{model}^{\text{student}}$ leverages the hard work used to train $\backslash \text{model}^{\text{teacher}}$.

Often

- $\backslash \text{model}^{\text{teacher}}$ is an existing model
 - trained at great expense by someone else
- $\backslash \text{model}^{\text{teacher}}$ is much larger (in number of parameters) than $\backslash \text{model}^{\text{student}}$

Reducing the cost of running a reasoning model

This is another use of Distillation

- Distill the knowledge of a large (high number of parameters) model \model^{teacher}
- Into a smaller model \model^{student}

Reasoning models

- with relatively small number (tens of billions) parameters
- have been distilled from much larger reasoning models (hundreds of billions parameters)
- with small decreases in performance

Thinking harder

The reasoning trace `\rat` is a sequences of thoughts.

How long should a model "think" ?

- what is the length (measured in tokens of the reasoning traces `\rat` ?

The initial approaches to "reasoning" favored

- long thoughts (thinking "harder" or "deeper")

So one approach to a better Reasoner is getting it to produce longer reasoning traces

Thinking budget

Reference

[s1: Simple test-time scaling \(https://arxiv.org/pdf/2501.19393\)](https://arxiv.org/pdf/2501.19393).

An [interesting approach \(https://arxiv.org/pdf/2501.19393\)](https://arxiv.org/pdf/2501.19393) is to force a Reasoner to obey a thinking budget for its reasoning traces

- measured in number of thoughts, or tokens
 - minimum
 - maximum

This approach would fall into the "search" class of using test-time compute.

Rather than training the Reasoner to obey a budget

- this approach **modifies the Inference loop**
 - the loop enforcing the auto-regressive behavior of the LLM
- to
 - force the model to continue inference if the reasoning trace is below budget
 - truncate inference when the budget is exceeded

Forcing an overly long reasoning trace to adhere to maximum length is straight-forward

- Truncate the trace
- Insert an `<eos>` token

When the reasoning trace produced is too short

- the modified inference loop
 - replaces the token
 - with a token sequence that causes the model to revise/extend the reasoning trace
 - e.g., "Alternatively", "Wait", "But"
 - thus causing the LLM to continue "thinking"

Here is what the result looks like:

Budget Forcing

How many r in raspberry?

Question

Let's break down the process of counting the letter 'r' in the word "raspberry" ...

- * First letter: 'r' - This is an 'r', count = 1.
- * Second letter: 'a' - Not an 'r', count remains 1 ...
- * Sixth letter: 'e' - Not an 'r', count remains 1.
- * Seventh letter: 'r' - This is an 'r', count = 2.
- * Eighth letter: 'y' - Not an 'r', count remains 2 ...

The number of 'r's in "raspberry" is 2.

Wait, let's re-read the question carefully. It asks "How many r in raspberry?" ... * r - a - s - p - b - e - r - r - y ... * First 'r' ... * Second 'r' ... * Third 'r' ... Count = 3 ...

Reasoning trace

My initial answer of 2 was incorrect due to a quick reading of the word. **Final Answer:** The final answer is **3**

Response

Figure 3. Budget forcing with s1-32B. The model tries to stop after "...is 2.", but we suppress the end-of-thinking token delimiter instead appending "Wait" leading **s1-32B** to self-correct its answer.

Attribution: <https://arxiv.org/pdf/2501.19393#page=4>

Code for enforcing a thinking budget

Here is some simplified code

- derived from the paper's [Github \(https://github.com/simplescaling/s1/tree/main?tab=readme-ov-file#vllm-with-budget-forcing\)](https://github.com/simplescaling/s1/tree/main?tab=readme-ov-file#vllm-with-budget-forcing)
- note some minor formatting differences
 - does not explicitly use `<think>` and `</think>` to denote reasoning trace
 - instead, uses `<|im_start|>think`
 - denotes this part of the "conversation" is "thinking mode"

Note

- the multiple uses of the LLM `generate` method
- to generate the next part of the output
 - auto-regressive loop

Code: Budget Forcing
creating the reasoning trace

```
# Constants ignore_str = "Wait" max_tokens_thinking_tmp = MAX_TOKENS_THINKING
# Generate the start of the reasoning trace: Change the Assistant's role to **think**
prompt += "<|im_start|>think" o = model.generate( prompt,
sampling_params=sampling_params ) # Increase length of reasoning trace until length is
at least MAX_TOKENS_THINKING if max_tokens_thinking_tmp > 0: for i in
range(NUM_IGNORE): # Num of times to skip stop token # Append the last extension to
the reasoning trace prompt += o[0].outputs[0].text # Insert a "Wait" prompt +=
ignore_str # Generate the next extension of the trace o = model.generate( prompt,
sampling_params=sampling_params ) # Reduce the remaining number of thinking tokens
to generate max_tokens_thinking_tmp -= len(o[0].outputs[0].token_ids) ...
```

Code: Budget Forcing creating the final answer

```
### Final answer ### # Append the last extension to the reasoning trace prompt +=  
o[0].outputs[0].text # You can also append "Final Answer:" here like we do for some  
evaluations # to prevent the model from just continuing to reason in its answer # when  
early exiting # Create the "answer", which follows the reasoning trace ... o =  
model.generate( prompt, sampling_params=sampling_params, ) print("With budget  
forcing:") # You will see that after the "Wait" in the reasoning trace it fixes its answer  
print(prompt + o[0].outputs[0].text)
```

Smarter not longer

At first glance

- longer reasoning trace should be preferred to a shorter trace
 - "deeper" reasoning

There is some empirical evidence that shows

- first preliminary response *can* often lead to correct response
- **but** the initial reasoning trace is often prematurely abandoned
 - the model "under-thinks" and tries something else if the first approach continues for too long

Why does a potentially successful initial reasoning trace get abandoned by the Reasoner ?

Perhaps it is in the training dataset

- hard problems in the training set have long but unsuccessful reasoning traces
- the model learns to abandon long traces

The problem is the inability to distinguish between

- long traces of hard problems that fail to be solved
- long traces that are needed for less-hard, solvable problems

How to think smarter

One approach is

- train a model to estimate the difficulty of a give task
- have the Reasoner adapt its test-time compute based on the difficulty
 - more/longer thoughts for harder tasks

Alternatively

- use a *Process Reward Model*
 - train a Reward Model to estimate
 - whether each step in the thought is advancing toward a good response
 - continue a thought only if the estimated reward is high

The training dataset for a Process Reward Model

- is expensive
- human labeling of the steps and rewards
 - reward limited to categorical (Positive/Negative/Neutral) versus continuous values

In [4]: `print("Done")`

Done

